主函数注解:

1.

```
0           0 RESUME                   0

1           2 LOAD_CONST               0 (<code object bubble_sort at 0x000001F86138E8D0, file "bubble_sort.py", line 1>)
            4 MAKE_FUNCTION            0
            6 STORE_NAME               0 (bubble_sort)
```

定义并绑定函数 bubble_sort

2.

```
18          8 LOAD_NAME                1 (__name__)
           10 LOAD_CONST               1 ('__main__')
           12 COMPARE_OP              40 (==)
           16 POP_JUMP_IF_FALSE       89 (to 196)
```

实现 if __name__ == "__main__": 的条件判断

3.

```
19          18 PUSH_NULL
            20 LOAD_NAME               2 (input)
            22 LOAD_CONST              2 ('请输入要排序的整数列表（用空格分隔）：')
            24 CALL                    1
            32 STORE_NAME              3 (raw)
```

读取用户输入并存入 raw

4. 解析输入字符串，构造 arr 列表

4.1

```
20          34 LOAD_NAME               3 (raw)
            36 LOAD_ATTR               9 (NULL|self + strip)
            56 CALL                    0
            64 POP_JUMP_IF_FALSE      36 (to 138)
```

条件判断部分

4.2

```
            66 LOAD_NAME               3 (raw)
            68 LOAD_ATTR              11 (NULL|self + split)
            88 CALL                    0
            96 GET_ITER
            98 LOAD_FAST_AND_CLEAR     0 (x)
           100 SWAP                    2
           102 BUILD_LIST              0
           104 SWAP                    2
       >>  106 FOR_ITER                10 (to 130)
           110 STORE_FAST              0 (x)
           112 PUSH_NULL
           114 LOAD_NAME               6 (int)
           116 LOAD_FAST               0 (x)
           118 CALL                    1
           126 LIST_APPEND             2
           128 JUMP_BACKWARD           12 (to 106)
       >>  130 END_FOR
           132 SWAP                    2
           134 STORE_FAST              0 (x)
           136 JUMP_FORWARD            1 (to 140)
```

实现 raw.split() 的迭代，并对每个元素调用 int(x)，把结果追加到新列表中

4.3

```
        >>  138 BUILD_LIST               0
        >>  140 STORE_NAME               7 (arr)
```

如果 raw.strip() 为假，则把 arr 设为一个空列表

5.

```
21          142 PUSH_NULL
            144 LOAD_NAME                8 (print)
            146 LOAD_CONST               3 ('原列表：')
            148 LOAD_NAME                7 (arr)
            150 CALL                     2
            158 POP_TOP
```

打印原列表

6.

```
23          160 PUSH_NULL
            162 LOAD_NAME                0 (bubble_sort)
            164 LOAD_NAME                7 (arr)
            166 CALL                     1
            174 POP_TOP
```

执行 bubble_sort(arr)，对列表进行冒泡排序

7.

```
25          176 PUSH_NULL
            178 LOAD_NAME                8 (print)
            180 LOAD_CONST               4 ('排序后：')
            182 LOAD_NAME                7 (arr)
            184 CALL                     2
            192 POP_TOP
            194 RETURN_CONST             5 (None)
```

打印排序后的列表并返回

8.

```
18      >>  196 RETURN_CONST             5 (None)
        >>  198 SWAP                     2
            200 POP_TOP

20          202 SWAP                     2
            204 STORE_FAST               0 (x)
            206 RERAISE                  0
ExceptionTable:
  102 to 130 -> 198 [2]
```

如果__name__不等于"__main__"，那么直接返回 None 并抛出异常信息

核心函数 bubble_sort 注解：

```
 1           0 RESUME                   0

 5           2 LOAD_GLOBAL              1 (NULL + len)
            12 LOAD_FAST                0 (arr)
            14 CALL                     1
            22 STORE_FAST               1 (n)

 6          24 LOAD_GLOBAL              3 (NULL + range)
            34 LOAD_FAST                1 (n)
            36 LOAD_CONST               1 (1)
            38 BINARY_OP               10 (-)
            42 CALL                     1
            50 GET_ITER
      >>    52 FOR_ITER                71 (to 198)
            56 STORE_FAST               2 (i)

 7          58 LOAD_CONST               2 (False)
            60 STORE_FAST               3 (swapped)

 9          62 LOAD_GLOBAL              3 (NULL + range)
            72 LOAD_CONST               3 (0)
            74 LOAD_FAST                1 (n)
            76 LOAD_CONST               1 (1)
            78 BINARY_OP               10 (-)
            82 LOAD_FAST                2 (i)
            84 BINARY_OP               10 (-)
            88 CALL                     2
            96 GET_ITER
      >>    98 FOR_ITER                42 (to 186)
           102 STORE_FAST               4 (j)

10         104 LOAD_FAST                0 (arr)
           106 LOAD_FAST                4 (j)
           108 BINARY_SUBSCR
           112 LOAD_FAST                0 (arr)
           114 LOAD_FAST                4 (j)
           116 LOAD_CONST               1 (1)
           118 BINARY_OP                0 (+)
           122 BINARY_SUBSCR
           126 COMPARE_OP              68 (>)
           130 POP_JUMP_IF_TRUE         1 (to 134)
           132 JUMP_BACKWARD           18 (to 98)

11     >>  134 LOAD_FAST                0 (arr)
           136 LOAD_FAST                4 (j)
           138 LOAD_CONST               1 (1)
           140 BINARY_OP                0 (+)
           144 BINARY_SUBSCR
           148 LOAD_FAST                0 (arr)
           150 LOAD_FAST                4 (j)
           152 BINARY_SUBSCR
           156 SWAP                     2
           158 LOAD_FAST                0 (arr)
           160 LOAD_FAST                4 (j)
           162 STORE_SUBSCR
           166 LOAD_FAST                0 (arr)
           168 LOAD_FAST                4 (j)
           170 LOAD_CONST               1 (1)
           172 BINARY_OP                0 (+)
           176 STORE_SUBSCR

12         180 LOAD_CONST               4 (True)
           182 STORE_FAST               3 (swapped)
           184 JUMP_BACKWARD           44 (to 98)

 9     >>  186 END_FOR

14         188 LOAD_FAST                3 (swapped)
           190 POP_JUMP_IF_FALSE        1 (to 194)
           192 JUMP_BACKWARD           71 (to 52)

15     >>  194 POP_TOP
           196 RETURN_CONST             5 (None)

 6     >>  198 END_FOR
           200 RETURN_CONST             5 (None)
```

```
1           0 RESUME                   0
```

函数入口

```
5           2 LOAD_GLOBAL              1 (NULL + len)
           12 LOAD_FAST                0 (arr)
           14 CALL                     1
           22 STORE_FAST               1 (n)
```

从全局命名空间取出 len 函数

加载局部变量 arr

用 1 个参数调用 len(arr)

将返回值存入局部变量 n

```
6          24 LOAD_GLOBAL              3 (NULL + range)
           34 LOAD_FAST                1 (n)
           36 LOAD_CONST               1 (1)
           38 BINARY_OP               10 (-)
           42 CALL                     1
           50 GET_ITER
      >>   52 FOR_ITER                71 (to 198)
           56 STORE_FAST               2 (i)
```

取出 range 函数

计算 n – 1

得到 range(n - 1)对象

取迭代器

开始外层 for 循环，每次从迭代器中取一个值给 i，如果结束则跳到 198

把当前轮次的循环变量存为局部变量 i

```
7          58 LOAD_CONST               2 (False)
           60 STORE_FAST               3 (swapped)
```

False 存入局部变量 swapped，本轮尚未发生交换

```
9           62 LOAD_GLOBAL              3 (NULL + range)
            72 LOAD_CONST               3 (0)
            74 LOAD_FAST                1 (n)
            76 LOAD_CONST               1 (1)
            78 BINARY_OP               10 (-)
            82 LOAD_FAST                2 (i)
            84 BINARY_OP               10 (-)
            88 CALL                     2
            96 GET_ITER
      >>    98 FOR_ITER                42 (to 186)
           102 STORE_FAST               4 (j)
```

取 range 函数

下界为 0

先算 n – 1

再算(n - 1) – i（上界）

调用 range(0, n - 1 - i)

取迭代器，开始内层循环

每次迭代产生一个 j，如果结束则跳到 186

将这一轮索引存入局部变量 j

```
10         104 LOAD_FAST                0 (arr)
           106 LOAD_FAST                4 (j)
           108 BINARY_SUBSCR
           112 LOAD_FAST                0 (arr)
           114 LOAD_FAST                4 (j)
           116 LOAD_CONST               1 (1)
           118 BINARY_OP                0 (+)
           122 BINARY_SUBSCR
           126 COMPARE_OP              68 (>)
           130 POP_JUMP_IF_TRUE         1 (to 134)
           132 JUMP_BACKWARD           18 (to 98)
```

取 arr[j]

计算 j + 1

取 arr[j + 1]

比较 arr[j] > arr[j + 1]

如果条件为真，跳到 134

否则回到 98，继续内层 for 的下一次迭代

```
11     >>  134 LOAD_FAST                0 (arr)
           136 LOAD_FAST                4 (j)
           138 LOAD_CONST               1 (1)
           140 BINARY_OP                0 (+)
           144 BINARY_SUBSCR
           148 LOAD_FAST                0 (arr)
           150 LOAD_FAST                4 (j)
           152 BINARY_SUBSCR
           156 SWAP                     2
           158 LOAD_FAST                0 (arr)
           160 LOAD_FAST                4 (j)
           162 STORE_SUBSCR
           166 LOAD_FAST                0 (arr)
           168 LOAD_FAST                4 (j)
           170 LOAD_CONST               1 (1)
           172 BINARY_OP                0 (+)
           176 STORE_SUBSCR
```

取得 arr[j + 1]和 arr[j]，并按交换顺序写回

取出 arr[j + 1]，压栈

取出 arr[j]，压栈

把其中一个值写回 arr[j]

另一个值写回 arr[j + 1]

从而交换两个索引对应的值

```
12          180 LOAD_CONST               4 (True)
            182 STORE_FAST               3 (swapped)
            184 JUMP_BACKWARD           44 (to 98)
```

把 True 赋给 swapped（有进行过交换）

跳回 98，继续 for 循环下一次迭代

```
 9      >>  186 END_FOR
```

结束循环

```
14          188 LOAD_FAST                3 (swapped)
            190 POP_JUMP_IF_FALSE        1 (to 194)
            192 JUMP_BACKWARD           71 (to 52)
```

取出当前趟是否有发生交换的变量

如果 swapped 为假（not swapped 为真），跳到 194。如果为真，跳到 52 继续循环

从外层循环 break 后返回

正常跑完所有外层循环后返回